A Cluster-Local Softmax Merge Unit for RISC-V Vector Clusters

Anonymous Author(s)

***Abstract*—Blockwise attention updates a compact (m, l, O) state after every tile. A general-purpose core pays scalar control and normalization costs for every row, even when vector hardware handles the output update. This paper presents a cluster-local Softmax Merge Unit (SMU) for this recurrence. The SMU uses the existing memory-mapped control plane and TCDM scratchpad interface of a Spatz RISC-V vector cluster. It therefore leaves the ISA and the main vector pipeline unchanged. The datapath evaluates the maximum, two exponential scales, reciprocal-based weights, and the weighted output-vector update. Across 16 measured (N, D) coordinates, the full SMU outperforms an RTL-aligned scalar-plus-RVV baseline by 1.42–10.14×. Concurrent execution with a streaming core workload preserves the SMU execution time within 0.05% at $(16, 64)$, while the core slows by 0.6%. The results show that a compact auxiliary engine can remove per-row merge overhead without requiring an ISA extension.**

***Index Terms*—online softmax, attention, RISC-V vector processor, scratchpad, accelerator**

I. Introduction

Blockwise attention maintains an online normalizer and a partial output state instead of materializing a full attention matrix [1], [2]. Each completed tile merges one scalar maximum, one scalar normalizer, and one output vector into the running state. This recurrence has little instruction level parallelism across its scalar steps, yet it executes once for every row and tile. A vector core can update O, but it still pays the scalar merge, normalization, command, and synchronization costs.

We study whether a small cluster-local engine can execute this state update without changing the processor instruction path. Spatz combines compact RISC-V vector units with a banked L1 scratchpad [3]. Its cluster integration offers an opportunity: an auxiliary unit can receive commands through existing memory-mapped registers and exchange state through TCDM. This interface isolates the mechanism from the decoder, vector register file, and vector functional units.

This paper makes three contributions. First, it implements a full mixed-scalar online-softmax merge datapath behind an existing MMIO/TCDM boundary. Second, it evaluates the design against scalar and RVV-assisted software paths across a measured scaling matrix. Third, it measures the internal state-machine phases and shared-TCDM interference, which identify the remaining vector streaming bottleneck and quantify coexistence with a core workload.

II. Selective-Offload Architecture

Proposed is a Scalar SMU paired with existing RVV: the SMU handles the state-dependent scalar recurrence, while the core retains the regular $O[D]$ vector update. This selective boundary moves normalization control across the accelerator interface and leaves the established vector datapath and software sequencing in place. Shared TCDM carries the handoff state, so the architecture composes an RTL-defined scalar engine with an unchanged RVV update path. The scalar unit owns quantities whose next value depends on the row's selected maximum and normalization state; RVV owns operations that repeat over independent vector elements. The handoff therefore preserves software-visible memory ordering while narrowing hardware specialization to the recurrence. Figure 1 shows this selective-offload boundary and its TCDM-mediated handoff.

A. Specialization Boundary

Online merge separates state selection and normalization from the regular elementwise vector update. For one row, the decomposition is

$$m_{\text{new}} = \max(m_{\text{old}}, m_{\text{tile}}), \tag{1}$$

$$s_{\text{old}} = l_{\text{old}} \exp(m_{\text{old}} - m_{\text{new}}), \tag{2}$$

$$s_{\text{tile}} = l_{\text{tile}} \exp(m_{\text{tile}} - m_{\text{new}}), \tag{3}$$

$$l_{\text{new}} = s_{\text{old}} + s_{\text{tile}}, \tag{4}$$

$$w_{\text{old}} = s_{\text{old}}/l_{\text{new}}, \quad w_{\text{tile}} = s_{\text{tile}}/l_{\text{new}}, \tag{5}$$

$$O_{\text{new}}[j] = w_{\text{old}} O_{\text{old}}[j] + w_{\text{tile}} O_{\text{tile}}[j], \quad 0 \le j < D. \tag{6}$$

The first five lines execute in Scalar SMU, while the final line executes in existing RVV. EXP and reciprocal blocks implement RTL approximations, so these equations describe the decomposition rather than an IEEE-exact arithmetic claim. The index constraint makes the handoff explicit: the SMU produces per-row weights and RVV applies them across D elements. These lines reduce each row to two weights before any vector element moves. The decomposition also separates row-level dependencies from element-level accumulation, which defines the architecture's scheduling boundary.

Scalar recurrence belongs on a state-dependent datapath because both exponential arguments depend on the selected maximum and the reciprocal depends on the newly accumulated length. A regular vector loop excels at independent $O[j]$ operations after weights are known, but it does not remove the serial decisions that produce those weights. The boundary therefore assigns control-sensitive work to Scalar SMU and keeps uniform multiply-add work on existing RVV. This division avoids duplicating vector functional units while matching each phase to its natural execution granularity. Both decisions serialize within a row, whereas elements within one O row share the resulting weights and expose regular

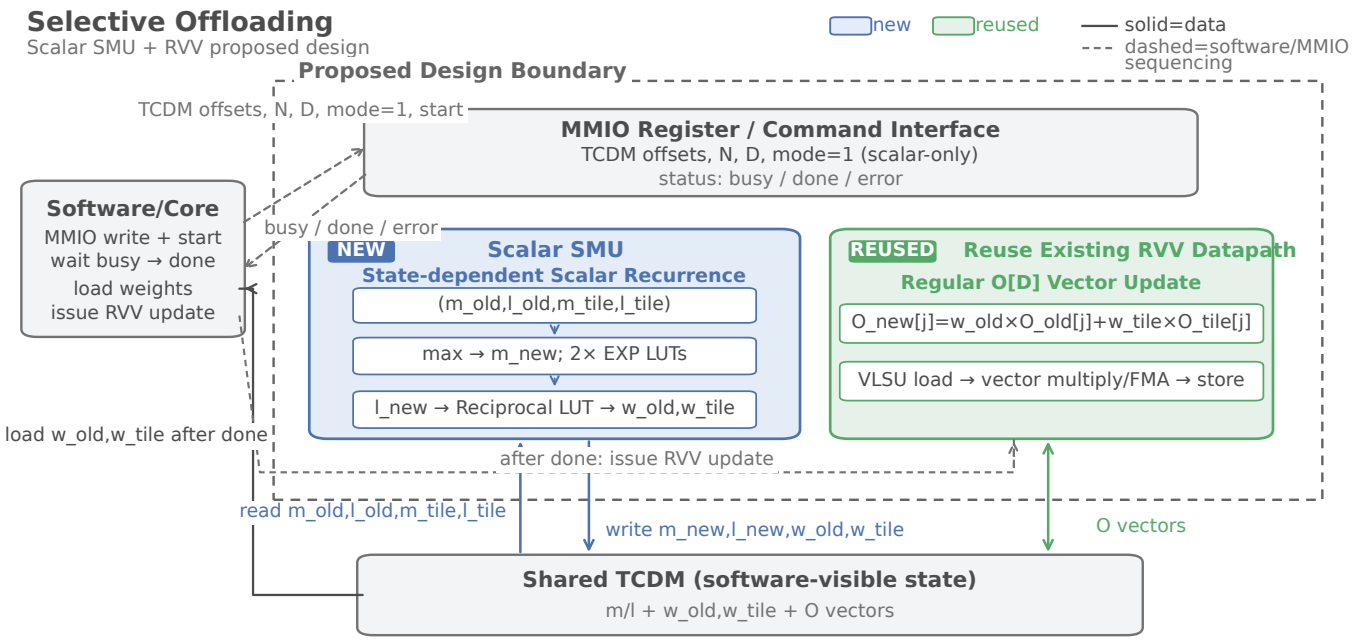

Fig. 1. Proposed architecture. Mode 1 directs the Scalar SMU to read scalar state and write m_{new}, l_{new}, w_{old}, and w_{tile} to shared TCDM. Software waits for completion, then invokes existing RVV for the $O[D]$ update; the diagram has no direct SMU-to-RVV link.

parallelism. Keeping this distinction explicit lets the core reuse its tuned VLSU and FMA sequence.

B. Scalar Datapath and Schedule

The Scalar SMU implements the recurrence with a compact scalar datapath. A comparator selects m_{new} and forms two deltas; two EXP approximation/LUT blocks transform those deltas into scaled factors. Fixed-point multipliers combine the factors with l_{old} and l_{tile}, an accumulator forms l_{new}, and a reciprocal approximation normalizes both scaled lengths. Weight generation then emits w_{old} and w_{tile} for TCDM writeback. The RTL places these operations in the scalar states. In mode 1, control exits after scalar writeback and never enters the engine's `UPDATE_VECTOR` state; Proposed performs the vector update on existing RVV. All intermediate values remain on the scalar path until writeback, so the datapath follows the recurrence's data dependencies. The resulting interface exports weights as ordinary TCDM state for the next phase.

RTL observer counts establish a fixed per-row scalar schedule: `LOAD_SCALAR` consumes 13 cycles, `COMPUTE_SCALAR` 1, `COMPUTE_WEIGHT` 1, and `STORE_SCALAR` 9, for 24 SMU-busy cycles per row. LOAD and STORE account for 22 of 24 cycles as TCDM communication states; the two compute states occupy the remaining two cycles. The 24-cycle count describes the scalar-only FSM schedule. It is neither the fitted $C_s = 90.28$ nor software-visible or end-to-end latency, which also includes command sequencing and completion polling. The observer records these states only for the four scalar states because mode 1 never enters `UPDATE_VECTOR`. The communication count reflects four reads plus four writes at state-machine handshake granularity, while the compute states represent combinational evaluations.

C. TCDM-Mediated RVV Handoff

Mode 1 makes the handoff state explicit in TCDM. For each row, the Scalar SMU reads four FP32 scalar inputs, m_{old}, l_{old}, m_{tile}, and l_{tile}, from their TCDM arrays. It computes the new scalar state and writes m_{new}, l_{new}, w_{old}, and w_{tile} back to TCDM for all rows. The output weights are software-visible arrays rather than private accelerator state, which gives the core a stable handoff point. Because the weight arrays share TCDM with the state, the SMU does not retain a private weight interface for the core. The same row indexing keeps scalar outputs aligned with subsequent vector rows.

Software sequences the two engines through status and memory. It programs the MMIO command, starts mode 1, polls the status until done, and then reads the two weight arrays from TCDM. After completion, software invokes existing RVV to load $O_{\text{old}}[D]$ and $O_{\text{tile}}[D]$, apply the weights, and store $O_{\text{new}}[D]$. The ordering gives RVV a completed, software-addressable weight state; no direct SMU-to-RVV path or weight-register bypass exists. Software remains the sequencing agent: it observes completion before exposing weights to RVV, so the core never consumes partially written rows. The RVV routine retains its existing load, multiply, FMA, and store sequence.

MMIO supplies command and status signals without changing the ISA. The cluster peripheral presents the software-selected operation to the Scalar SMU, while the engine contributes an independent TCDM requester to the shared interconnect. Existing core ports continue to serve RVV loads

and stores, so the handoff traverses shared TCDM rather than a private engine link. This structure changes neither instruction decoding nor the RVV datapath. The interconnect arbitrates both requester classes through existing TCDM machinery, while MMIO remains the control-plane boundary. This composition adds no decoder instruction or custom vector operand.

This boundary fixes the architecture evaluated next: Scalar SMU removes state-dependent scalar recurrence work, and existing RVV retains the regular vector update through a software-mediated TCDM handoff. The Evaluation section measures the resulting cycle behavior and then separates scaling, model-derived configurations, and standalone hardware evidence. This framing lets the next section attribute cycle changes to the scalar recurrence while retaining a fixed vector implementation in the primary comparison. It also keeps standalone evidence separate from measured cycles.

III. Evaluation

Selective scalar offloading retains the low vector cost of existing RVV and yields a $4.81\times$ geometric-mean speedup in measured cycles over B2R. The Proposed design combines a Scalar SMU with existing RVV, while Full serves only as the Full-Offload Ablation. B2R supplies the head-to-head reference because it already performs the vector update with existing RVV. This reference makes scalar recurrence offload the measured question and keeps Full's role diagnostic. The primary result therefore concerns selective offload with the vector path held fixed in the comparison.

A. Experimental Setup

We evaluate FP32 state on a Spatz cluster with the (m, l, O) arrays resident in TCDM. The simulator runs RTL through Verilator, and every reported cycle count comes from the measured RTL/Verilator run. Each implementation uses the same generated state and command shape, so cycle differences reflect the execution path. The state includes old and tile scalar values together with output vectors, and the benchmark places source and destination arrays in TCDM before launch. We use one cluster configuration and one launch protocol across roles. RTL/Verilator cycles include the software-visible command and completion work, so each role exposes the path that its implementation actually executes.

We compare four roles. B1 runs the scalar software context. B2R runs the scalar recurrence in software and uses existing RVV for the vector update, making it the primary strong baseline. Proposed combines Scalar SMU with existing RVV. Full runs the Full-Offload Ablation, which moves the vector update into the SMU as well. We retain only outputs that pass the correctness gate; incomplete or failing cases do not enter a primary result. B1 supplies context for total scalar work but supplies no primary speedup. B2R keeps the vector loop on the existing RVV path, so its cycle count establishes the cost after vectorization. Proposed moves only the scalar recurrence across the SMU boundary, whereas Full moves both recurrence and vector update. This role assignment keeps each comparison tied to one architectural change.

We use Yosys/ABC with the Nangate45 library to collect standalone hardware evidence. An area-oriented `abc -fast` mapping reports formal cell counts and mapped area. A separate timing-driven mapping reports critical delay and an estimated standalone Fmax. The two flows answer different questions, so we keep their cell and area outputs separate from timing outputs. The available records provide no cluster PPA or power measurement. The area flow supports a standalone size comparison, while the timing flow supports a standalone timing estimate under its own constraints. We never combine a timing-flow cell count with the area-flow result. The evidence therefore supports the SMU-local comparison and leaves cluster-level PPA and power outside the reported scope.

B. Progressive Baseline

B1 to B2R isolates the contribution of RVV vectorization. B1 remains in Table I as cycle context, while B2R supplies the primary strong baseline for every speedup. This comparison identifies the cost that existing RVV removes before any SMU offload. The B1 column lets readers see how the scalar context grows with each model-derived shape, but it carries no speedup column. B2R preserves the same state representation and command setup while using the existing vector instructions, which makes the next comparison isolate the recurrence itself.

Every primary speedup divides B2R cycles by Proposed cycles. The ratio attributes improvement to scalar recurrence offload after RVV handles the vector update. Full appears only to quantify the cost and benefit of moving that vector update as an ablation. A common denominator keeps the interpretation stable across shapes and prevents a scalar-context comparison from becoming a headline result. The reported ratios thus measure the incremental contribution of Scalar SMU under the same RVV vector path.

C. Scaling and Cost Decomposition

To explain the cycle trend, we fit $C = C_0 + C_s N + C_v N D$ to each design. Here N denotes rows, D denotes vector dimension, C_0 captures fixed work, C_s captures per-row scalar recurrence work, and C_v captures per-element vector work. The fitted coefficients summarize RTL/Verilator cycle trends; they do not equal FSM busy-cycle counts. Figure 2 presents the fit and uses a logarithmic scale for C_s. The decomposition treats N and D as independent shape variables, so each coefficient summarizes a different growth direction. C_0 absorbs launch and loop-independent work, whereas C_s and C_v describe fitted contributions across the measured coordinates. None of these coefficients is a direct FSM state counter.

B2R fits $C_s/C_v = 1594.23/2.09$, whereas Proposed fits $90.28/2.13$. Proposed therefore cuts the scalar coefficient by $17.66\times$ while increasing the vector coefficient by only 1.88%. The near-constant C_v shows that Scalar SMU offload removes recurrence work after RVV vectorization without changing the existing vector path. The fitted scalar term accounts for the large difference between the two paths, while the fitted vector term stays within the same scale. This separation links

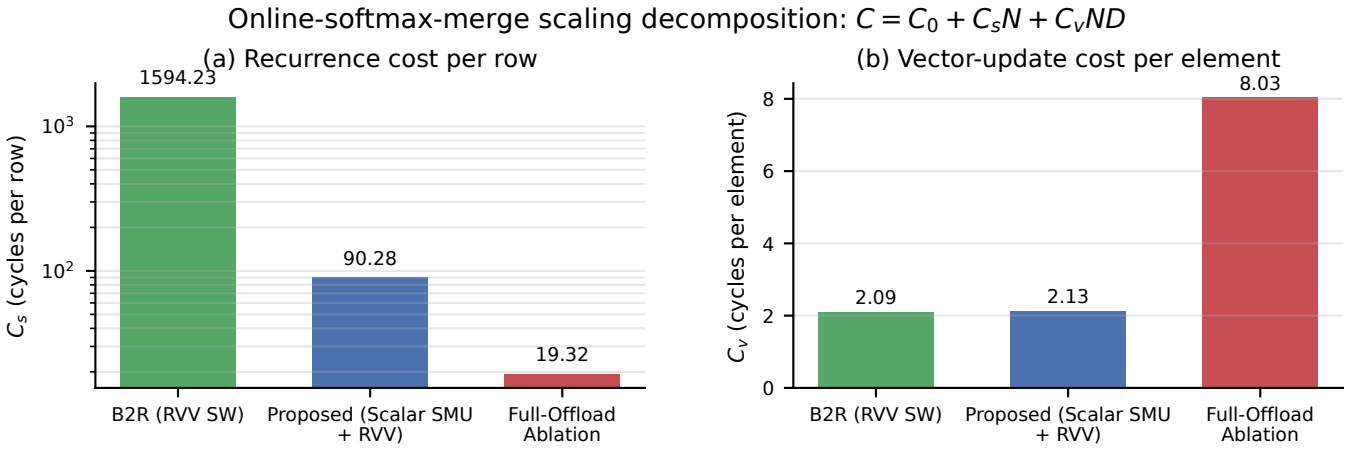

Fig. 2. Fitted C_s and C_v from RTL/Verilator cycles for B2R, Proposed, and Full. The C_s axis uses a log scale. The fitted terms describe cycle trends and exclude wall-time and FSM-busy-cycle interpretations.

the measured speedup to the specific recurrence work that Proposed moves off the core.

Full fits $C_s/C_v = 19.32/8.03$, and its C_v is 3.78× Proposed's. Full therefore reduces the scalar coefficient further but pays a larger vector coefficient when it absorbs the vector update. The decomposition points to the selective tradeoff: low C_s with low C_v gives Proposed its measured advantage. Full quantifies the cost of extending scalar offload into the vector path, so the ablation separates the two sources of cycle change. Proposed keeps the lower vector coefficient while retaining the scalar reduction that matters for the primary baseline.

D. Model-Derived Configurations

We next evaluate one attention-position shape per model. For each shape, we generate a deterministic merge state with FP32 arrays in TCDM and measure the four roles under the same RTL/Verilator flow. The selected shapes are BERT $(N, D) = (12, 64)$, Mistral $(32, 128)$, and Qwen14B $(40, 128)$. Table I reports the resulting cycles, with B1 included as context. Each row represents one attention-position merge state, so the table exercises the recurrence at a model-derived (N, D) shape. Deterministic state generation keeps the inputs aligned across roles and makes the measured cycle columns directly comparable. The table reports merge cycles only and uses no end-to-end inference schedule.

The Proposed/B2R speedups are 5.04× for BERT, 4.62× for Mistral, and 4.77× for Qwen14B. Their geometric mean is 4.81×, matching the section result. These ratios keep the vector update in both compared paths and isolate scalar recurrence offload across model-derived dimensions. The three ratios remain close because each workload uses the same role definitions and the same vector path in Proposed and B2R. The geomean therefore summarizes a consistent selective-offload effect across the tested model dimensions rather than a single favorable shape.

Qwen72B's selected shape exceeds local TCDM capacity, so the benchmark has no cycle result for that shape and this boundary closes the measured feasible scope.

E. Hardware Cost and Full-Offload Ablation

Table II separates area evidence from timing evidence. The area flow uses formal cell count and mapped area from `abc -fast`; the timing flow uses a separate timing-driven mapping for critical delay and standalone Fmax. We keep timing-flow cell and area outputs out of the area comparison so the two mappings answer their intended questions. The table consequently reports formal cells and area from one flow, then delay and estimated Fmax from the other. This separation preserves the meaning of each column and prevents a timing result from changing the area comparison.

Proposed maps to 73,505 cells and 77,103.3 Liberty units with a 12.34 ns critical delay and an estimated standalone Fmax of 81.02 MHz. Full maps to 107,372 cells and 114,713.3 Liberty units with a 13.20 ns critical delay and an estimated standalone Fmax of 75.74 MHz. Full increases standalone mapped area by 48.8% over Proposed. The Proposed row therefore combines the smaller formal mapped design with the shorter timing-flow delay in this standalone evidence. Full exposes the additional mapped logic required when the vector update joins the offload path.

Figure 3 places B2R at 1.00× speedup with zero SMU increment. Proposed reaches 4.81× at 77.103 k mapped area, and Full reaches 2.00× at 114.713 k. B2R's zero SMU increment denotes the software baseline and does not denote zero cluster area. The x-coordinate tracks standalone SMU mapped area, while the y-coordinate tracks measured cycles

TABLE I

MODEL-DERIVED ONE-POSITION SHAPES AND MEASURED RTL/VERILATOR CYCLES. VALUES DESCRIBE THE MERGE MICROBENCHMARK, NOT END-TO-END INFERENCE. B1 PROVIDES CONTEXT AND HAS NO SPEEDUP COLUMN.

Workload	(N, D)	B1 context	B2R	Proposed	Full ablation	B2R/Proposed
BERT	$(12, 64)$	271,718	20,785	4,128	7,704	5.04×
Mistral	$(32, 128)$	1,401,702	59,500	12,866	34,728	4.62×
Qwen14B	$(40, 128)$	1,780,547	75,027	15,733	43,206	4.77×
Geomean	–	–	–	–	–	4.81×

TABLE II

STANDALONE MAPPED HARDWARE EVIDENCE. THE AREA AND TIMING COLUMNS USE SEPARATE MAPPINGS; TIMING-FLOW CELLS AND AREA DO NOT ENTER THE AREA FLOW. FMAX IS A PRE-LAYOUT STANDALONE ESTIMATE.

Design	Mapped cells (area flow)	Mapped area (Liberty units)	Critical delay (timing flow)	Est. standalone Fmax
Proposed	73,505	77,103.3	12.34 ns	81.02 MHz
Full-Offload Ablation	107,372	114,713.3	13.20 ns	75.74 MHz

relative to B2R. The Full point shows how extra vector offload changes this standalone tradeoff, and workload crosses show variation across all three model-derived shapes in the measured comparison.

Using a common 81.0186 MHz assumption yields a 2.40× Proposed/Full derived throughput ratio for the two standalone designs. The same frequency gives a 3.57× incremental standalone-SMU area-efficiency ratio. These A1/A2-only derived comparisons do not measure system throughput or cluster efficiency. Cluster area, cluster timing and Fmax, physical area, signoff, power, and energy remain unavailable. The selective tradeoff is clear: Scalar SMU plus existing RVV preserves low vector cost while Full spends area to offload the vector update. The frequency assumption lets us compare the two standalone designs on one common scale, but it cannot turn mapped evidence into a system measurement. The conclusion stays within A1/A2 evidence: selective scalar offload retains vector cost discipline while Full trades additional standalone area for vector-path offload in this ablation.

IV. DISCUSSION

A. The design targets a specific granularity

The SMU receives a complete merge command, so each launch carries a fixed configuration and completion cost. The measurements show that this choice works best when several rows amortize that cost. At $(1, 1)$, only 29 SMU busy cycles execute inside a 1,124-cycle end-to-end operation. At $(16, 64)$, the engine executes 8,534 busy cycles inside 9,556 end-to-end cycles. This change explains why the engine remains useful for matrix shapes that contain many short scalar recurrences, even when each recurrence has a compact vector.

The command boundary also defines a clear deployment path. A tiled attention kernel can retain its QK and PV computation on an existing vector or matrix engine, place the partial (m, l, O) state in local scratchpad memory, and submit merge commands at tile completion. The current prototype serializes one command, but independent rows and heads expose natural opportunities for multiple engines. A future front end can batch commands or use a queue to reduce the small-workload overhead that the current MMIO protocol reveals.

B. The evaluation distinguishes evidence layers

The paper uses three measurement layers because each answers a separate question. RTL end-to-end cycles establish the software-versus-engine result. FSM counters establish where the engine spends active cycles. The concurrency experiment establishes whether a co-running stream alters those component times. The combination gives a stronger explanation than a speedup number alone: the scalar work leaves the core, the vector loop becomes dominant, and the shared scratchpad introduces measurable yet small interference for the tested stream.

The numerical results also have two layers. The host model tests the selected LUT interpolation and reciprocal construction against the complete recurrence. The RTL benchmark tests the implemented fixed-point data path against its RTL-aligned software reference. Reporting both values prevents the lower model error from becoming an end-to-end RTL claim. The explicit validation paths serve the same purpose. They show that the engine rejects malformed commands instead of silently producing an output.

C. Proxy data guides follow-up work

The generic logic and toggle measurements guide architectural choices but do not replace physical implementation. The resource decomposition assigns most generic cells to the vector arithmetic and data path, while the exponential and reciprocal blocks occupy smaller independent scopes. The toggle traces show a lower count for B3 at the largest sampled coordinate because B3 completes the operation in fewer target cycles. These observations motivate vector-path optimization and physical synthesis. They do not establish an area, frequency, power, or energy advantage.

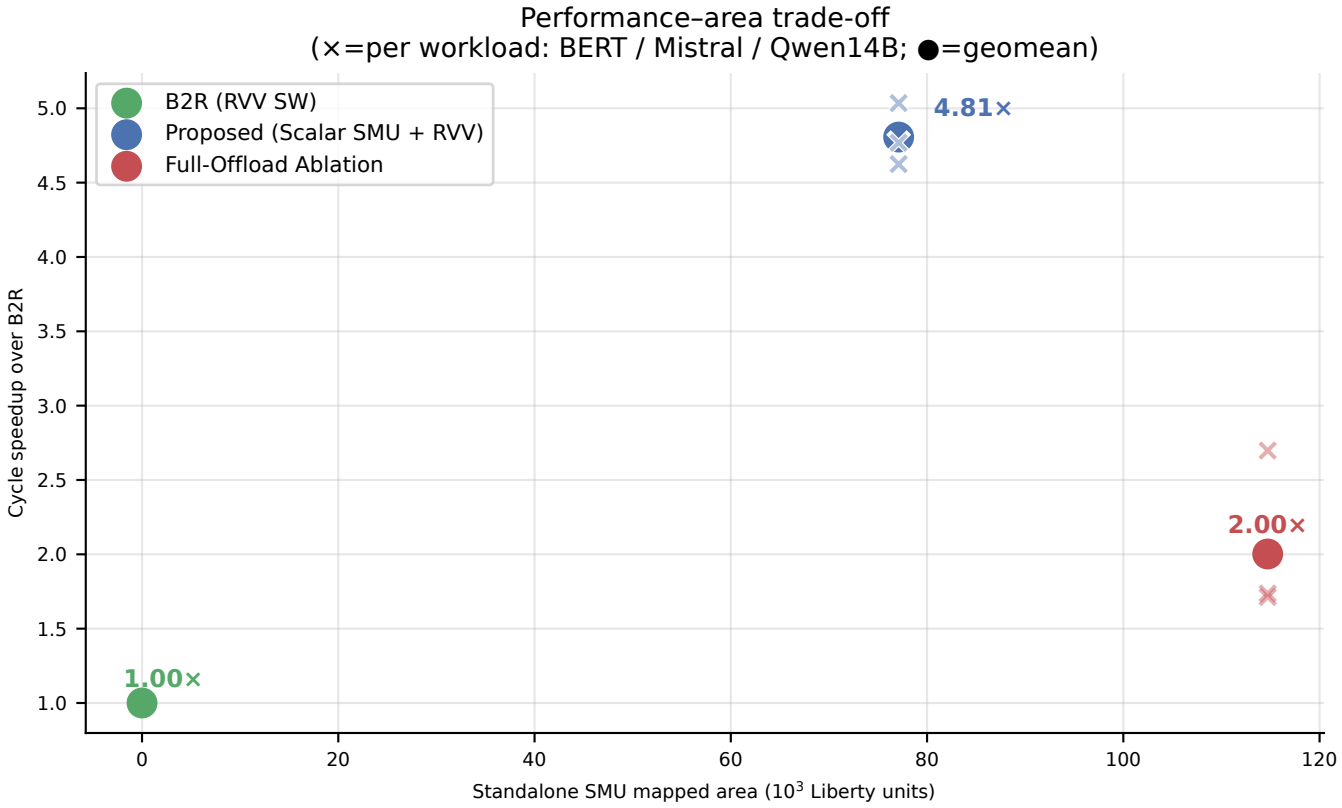

Fig. 3. Standalone SMU mapped area on the x-axis and frequency-independent measured-cycle speedup over B2R on the y-axis. B2R's zero SMU increment does not denote zero cluster area. Circles mark geomeans and crosses mark workloads; the plot makes no cluster-area claim.

V. Limitations and Related Work

Online normalization provides the recurrence that enables streaming softmax state [1]. IO-aware attention algorithms use this structure to process tiles without materializing full score matrices [2]. The SMU complements these algorithms at the cluster level. It accelerates the merge state after a tile; it does not implement QK multiplication, PV multiplication, masking, scheduling, or an end-to-end attention runtime.

Spatz supplies compact vector execution and a shared scratchpad [3]. Our design adds a TCDM master beside this execution path instead of introducing a vector instruction. The evaluation compares RTL-aligned software paths, not an optimized CPU, GPU, or production attention kernel. The Yosys and toggle numbers remain logic-activity proxies. A physical technology library, timing constraints, and power analysis would be required for PPA claims.

The approximation range also bounds the current result. The exponential LUT saturates below -8 and above zero, and the input validation excludes NaN, infinity, and subnormal values. The measured numerical cases cover mixed scalar inputs, a delta sweep, and a merge-chain workload. They establish behavior for those cases; they do not provide a universal error bound over all attention distributions. A future design can add wider approximation ranges, refinement steps, or IEEE floating-point datapaths when the target workload requires them.

VI. Conclusion

The SMU moves online-softmax merge state updates into a compact TCDM-connected engine while preserving the Spatz ISA and vector pipeline. Measured B3 results outperform the RTL-aligned RVV-assisted baseline at all 16 scaling coordinates and retain component throughput during a shared-TCDM experiment. The measured FSM phases identify vector streaming and command overhead as the next design targets. These results establish cluster-local merge offload as a practical building block for future tiled-attention systems.

References

[1] M. Milakov and N. Gimelshein, "Online normalizer calculation for softmax," *arXiv preprint arXiv:1805.02867*, 2018.

[2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 16 344–16 359.

[3] M. Cavalcante, M. Perotti, S. Riedel, and L. Benini, "Spatz: Clustering compact risc-v-based vector units to maximize computing efficiency," *arXiv preprint arXiv:2309.10137*, 2023.